



TXW82x LCD FAQ



珠海泰芯半导体有限公司

Zhuhai Taixin Semiconductor Co., Ltd

珠海市高新区港湾一号科创园港 11 栋 3 楼

保密等级	A	TXW82x LCD FAQ	文件编号	TX-0000
发行日期	2026-07-15		文件版本	V1.3

责任与版权

责任限制

本文档内容仅供参考。珠海泰芯半导体有限公司（以下简称“泰芯”）未对本文档信息的准确性或完整性作出任何明示或暗示的陈述或保证，且不对使用本文件信息所产生的后果承担任何责任。

泰芯保留在不另行通知对本文件进行修改、改进或其他变更的权利。

泰芯没有为客户产品的应用、设计验证提供协助的义务。客户应自行对其应用的设计、验证和测试负责，并确保其应用符合所有法律、法规及安全相关的要求。

如因客户未遵守本协议导致任何损害、费用、损失及/或责任，泰芯不承担任何责任；同时，客户应全额赔偿因客户未遵守本协议而给泰芯造成的任何损害、费用、损失及/或责任。

版权声明

未经泰芯书面同意，任何一方不得为商业目的修改、改编、变更、翻译或基于本文件创建衍生作品。

未经泰芯书面同意，任何一方不得向第三方披露或分发本文件中提及的任何部分或全部源代码、SDK、二进制文件和目标代码。

任何一方不得修改、反向工程、反汇编、反编译或以其他方式试图发掘任何非源代码部分的 SDK，包括但不限于预编译的二进制文件和目标代码。

此外，任何可能侵犯泰芯或其他知识产权所有者权利的行为也均被严格禁止。

对于实施上述侵权行为的主体，泰芯有权依据中华人民共和国法律或其他可能适用的法律、国际条约采取必要的法律措施，包括但不限于对侵权人提起诉讼或仲裁，或申请法律强制措施。

珠海泰芯半导体有限公司

2024 年 09 月 24 日

	珠海泰芯半导体有限公司 Zhuhai TaiXin Semiconductor Co., Limited	珠海市高新区港湾一号科技园港 11 栋 3 楼
---	---	-------------------------

版权所有 侵权必究

Copyright © 2026 by TaiXin All Rights Reserved

保密等级	A	TXW82x LCD FAQ	文件编号	TX-0000
发行日期	2026-07-15		文件版本	V1.3

修订记录

日期	版本	描 述	修订人
2026-07-15	V1.3	增加 mcu-spi 屏幕帧间命令等修改	Ethan
2026-05-23	V1.2	添加 mipi 屏幕设置, rgb 屏幕抖动处理操作等	Ethan
2025-10-09	V1.1	初始版本	Ethan

	珠海泰芯半导体有限公司 Zhuhai TaiXin Semiconductor Co., Limited	珠海市高新区港湾一号科创园港 11 栋 3 楼
---	---	-------------------------

版权所有 侵权必究

Copyright © 2026 by TaiXin All Rights Reserved

保密等级	A	TXW82x LCD FAQ	文件编号	TX-0000
发行日期	2026-07-15		文件版本	V1.3
目录				
TXW82x LCD FAQ 1				
1. SPI 屏幕驱动配置讲解 1				
1.1. 简要说明 1				
1.2. SPI IO 配置 1				
1.3. LCDC 参数配置 2				
1.4. 屏幕寄存器初始化 3				
1.5. 检查初始化调用情况 4				
1.6. 运行 lvgl benchmark 测试出图 6				
2. MIPI-DSI 屏幕驱动配置讲解 7				
2.1. 简要说明 7				
2.2. DSI IO 配置 7				
2.3. MIPI-DSI 接口 LCDC 参数配置 8				
2.4. MIPI-DSI 屏幕调试技巧 10				
3. SPI 屏幕 CLK 异常停止，在 4 线 SD 卡带 FEM 应用中注意 11				
4. LCD 屏幕抖动 12				
4.1. dclk 补全设置 12				
4.2. 缩短帧间隔设置 12				
4.3. 修改 dma burst 12				
4.4. 添加 osd fifo 13				
4.5. OTA 出现抖动 13				
		珠海泰芯半导体有限公司 Zhuhai TaiXin Semiconductor Co., Limited	珠海市高新区港湾一号科技园港 11 栋 3 楼	
版权所有 侵权必究 Copyright © 2026 by TaiXin All Rights Reserved				

保密等级	A	TXW82x LCD FAQ	文件编号	TX-0000
发行日期	2026-07-15		文件版本	V1.3
4.6. 大屏 video+osd 频繁解码出现抖动的情况13 5. MCU 和 SPI 屏幕利用外部中断做 TE 同步13 5.1. 简要说明13 6. MCU 和 SPI 屏幕，增加帧间命令操作15 7. MCU 屏幕，WR 信号跳变不及时导致发送出错16 8. MCU 屏幕，ID 读取操作设置17 9. LCD 录屏模块，检查显存操作18 10. DITHER 模块使用介绍18 11. SCALE2 SCALE3 缩放模块使用介绍19 12. SCALE2 buf 空间申请规则介绍20 13. OSD 层透明色设置21 14. VIDEO 层插黑操作22				
		珠海泰芯半导体有限公司 Zhuhai TaiXin Semiconductor Co., Limited	珠海市高新区港湾一号科创园港 11 栋 3 楼	
版权所有 侵权必究 Copyright © 2026 by TaiXin All Rights Reserved				

1. SPI 屏幕驱动配置讲解

1.1. 简要说明

相比之前 81x 芯片 82x lcdc 模块进行了升级可以直接支持 spi 屏幕，不必在占用 spi 总线。
配置一个 spi 屏幕大致步骤，配置 io，lcdc 参数，替换 spi 屏幕驱动，编译调试出图。

Note: 以 release sdk txw82x_fpv_2.7.07-37092_beta 版本为基础介绍。

1.2. SPI IO 配置

SPI 屏幕常见控制引脚有：CS ， SCL ， SDA， DC ， TE， RES 等引脚， 分别作用如下：

- CS - 片选控制引脚
- SCL - 时钟信号输出脚
- SDA - 数据命令输出信号脚
- DC - 命令数据选择引脚
- TE - 防撕裂引脚， 出现撕裂的时候用于同步刷新
- RES - 复位引脚

在目前 SDK 中以上引脚, 如果定义了 PIN_FROM_PARAM 的话, 需要到 project/ config.cfg 中去配置对应 IO， 可以参考下图。

```
PIN_LCD_RESET      = PB_11
//spi_cs
LCD_VS_CS          = PB_13
//spi_dc
LCD_HS_DC          = PB_9
//spi_scl
LCD_DOTCLK_RWR     = PB_12
//spi_sda
LCD_D0             = PB_10
```

如果没有定义 PIN_FROM_PARAM, 只需要在 project_config.h 中修改即可, 可以参考下图。

```

435  #if (LCD_ST7789V2_SPI_EN ==1)|| (LCD_ST7735P3_80x160_EN==1)
436
437  #define PIN_LCD_RESET  PB_11 //lcd_res
438  #define DOTCLK_RWR  PB_12    // spi_scl
439  #define DE_ERD  255
440  #define VS_CS  PB_13         // spi_cs
441  #define HS_DC  PB_9          // spi_dc
442  #define LCD_TE  255          // lcd_te
443
444  #define LCD_D0  PB_10         // spi_sda

```

Note: 要注意芯片 IO 电压问题，譬如上图中用到的 PB9~PB13 是 VCC1 电压，开发板默认接的 1.8V。而 LCD 的 VDD 目前接的是 3.3V，**所以 VCC1 改接成 3.3V，避免通信失败。**

1.3. LCDC 参数配置

对 82x 芯片的 LCDC 模块配置，在 lcdstruct 结构体中，如果你要新加一份新的驱动芯片，可以模仿 st7735 的配置另 copy 一份.c 文件。以下介绍几项重要配置。

```

264  lcddev_t  lcdstruct = {
265      .name = "st7735P3",
266      .lcd_bus_type = LCD_BUS_SPI4,
267      .bus_width = LCD_BUS_WIDTH_1,
268      .color_mode = LCD_MODE_565,
269      .osd_scan_mode = LCD_ROTATE_270, //LCD_ROTATE_270
270      .scan_mode = LCD_ROTATE_90, //rotate 90
271      .te_mode = 0xff, //te mode, 0xff:disable
272      .colrarray = 0, //0:_RGB_ 1:_RBG_ 2:_GBR_ 3:_GRB_ 4:_BRG_ 5:_BGR_
273      //f(wr) = source_clk/div/2
274      //f(wr) >= screen_w * screen_h * clk_per_pixel * 60
275      .pclk = 40000000,
276      .even_order = 0,
277      .odd_order = 0,
278      .lcd_data_mode = (0<<31) //data inversion mode
279                      (2<<24) //data compress mode
280                      (1<<20) //fifo mode
281                      (0<<17) //output cycle 2 shift direction
282                      (0<<12) //output cycle 2 shift bit
283                      (0<<11) //output cycle 1 shift direction
284                      (0<<6) | //output cycle 1 shift bit
285                      (0<<5) | //output cycle 0 shift direction
286                      (8<<0), //output cycle 0 shift bit
287      .screen_w = 128,
288      .screen_h = 160,
289      .video_x = XOFFSET,
290      .video_y = YOFFSET,
291      .video_w = 80,
292      .video_h = 160,
293      .osd_x = XOFFSET,
294      .osd_y = YOFFSET,
295      .osd_w = 80, // 0 : value will set to video_w , use for 4:3 LCD +16:9 sensor show UPDOWN BLACK
296      .osd_h = 160, // 0 : value will set to video_h , use for 4:3 LCD +16:9 sensor show UPDOWN BLACK
297      .init_table = st7735P3_register_init_tab,
298      .frame_table = NULL,
299      .clk_per_pixel = 2,
300
301      .pclk_inv = 1,
302
303
304      .vlw = 0, // 0,
305      .vbp = 0, // 12,
306      .vfp = 0, // 12,
307      .hlw = 0, // 1,
308      .hbp = 0, //14,
309      .hfp = 0, //13,
310
311
312      .de_en = 1,
313      .vs_en = 1,
314      .hs_en = 1,
315      .de_inv = 0xff,
316      .hs_inv = 0,
317      .vs_inv = 0,
318

```

1) 总线相关配置默认跟 st7735 一致:

- lcd_bus_type (总线类型), bus_width (总线宽度), color_mode (颜色模式)

2) 旋转角度配置:

- osd_scan_mode (osd 层 lvgl 的旋转角度), 按需配置。
- scan_mode (video 层的旋转角度特殊, 谨慎使用), 按需配置。

3) 总线时钟速率:

- pclk (spi 写入时钟频率), 由系统时钟分频而来, 根据 lcd 驱动芯片去适配。

4) lcd 有效显示尺寸设置

- screen_w/h: 需要按**实际显示范围**来设置。
- video_w/h 和 osd_w/h: 分别是 video 和 osd 层有效显示宽高, 按**实际显示范围**设置, **不能超过 screen_w/h**。
- video_x/y 和 osd_x/y: 分别是 video 和 osd 层偏移, 有需要的可以填没有的就默认 0, **确保所要显示的图像信息都在 screen_w/h 内, 不能超过 screen_w/h 的显示范围**。

5) init_table 是 lcd 寄存器初始化表, frame_table 帧间命令暂时可以不用, 需要的可以参考后面章节添加, 其余为提及部分跟 st7735 配置一致。

Note: 思维发散一下, 相同驱动芯片会遇到驱动不同 size 屏幕, 每次都需要改一下 2a, 2b 设置显示范围。是否可以不改初始化表, 可以通过 lcdc 偏移操作, screen_w/h 设置成驱动芯片显存范围, video_w/h 和 osd_w/h 设置有效显示范围, video_x/y 和 osd_x/y 设置 XOFFSET 和 YOFFSET 偏移, 参数是由**显存宽高和有效宽高计算得来**, 按照差值除 2 这个逻辑计算。设置后 lcdc 本身会做偏移, **所以不用在初始化 lcd 寄存器的时候设置 2a, 2b**。

```
9  #define LCD_W 80
10 #define LCD_H 160
11
12 #define XOFFSET ((128-LCD_W)/2)
13 #define YOFFSET ((160-LCD_H)/2)
```

1.4. 屏幕寄存器初始化

init_table, 这部分需要根据屏幕驱动 IC 来配置, 一般可以直接要求供应商提供! 但也会存在供应商给的配置文件不对无法点亮, 需要使用者有一定的 datasheet 阅读能力检查设置是否正常。

```

34  uint8_t st7735P3_register_init_tab[][2] = {
35
36      #if 1
37          CMD(0x11),
38          DLY( 120 ),
39  > #if 1...
122  #endif
123      CMD( 0x3A), //65k mode
124      DAT( 0x05),
125
126      /* Display Inversion 0n */
127      //CMD(0x21),
128
129      CMD( 0x29), // display on
130
131      CMD( 0x2A), // x address set
132      DAT( 0x00),
133      DAT( 0x00),
134      DAT( 0x00),
135      //DAT( 0x7F),
136      DAT( 0x83),
137
138      CMD( 0x2B), // y address set
139      DAT( 0x00),
140      DAT( 0x00),
141      DAT( 0x00),
142      //DAT( 0x9F),
143      DAT( 0xA1),
144
145      CMD( 0x2C), // write memory perpare
146
147      END

```

上图中展示的就是 LCD 寄存器初始化表：

隐藏部分是 porch/gate/vcom/lcm/vdv/vrh/power/gamma 等相关配置。（需要深入研究的可以参考 lcd 驱动芯片 datasheet）。

- 0x11, sleep out 退出睡眠，可以放置在最开始部分，随后加一定延时用于稳定。
- 0x3a, pixel format, 像素颜色格式设置，设置成 rgb565 跟 lcdc 模块一致。
- 0x21, display inversion, 反转显示模式，退出用 0x20，重复无设置无作用。
- 0x29, display on, 打开 lcd 显示，显存内容会被激活显示。
- 0x2a, 0x2b, 行列地址设置，用于设置显示有效区（这里设置成最大显存，lcdc 做偏移）。
- 0x2c, memory write, 写显存命令，发送显存数据前使用。
- END, 结束标识，用于判断结束。

1.5. 检查初始化调用情况

这部分由于部分新 sdk 释放冲忙，合并的时候未将初始化部分添加进去，主要检查以下部分：

- 1) app_lcd.c 文件下 lcd_hardware_init 函数分支是否有对 LCD_BUS_SPI4 类型做判断。

```

170 // 注意,这部分不同屏以及硬件设计不一样,需要修改
171 void lcd hardware_init(uint16_t *w, uint16_t *h, uint8_t *rotate, uint16_t *screen_w, uint16_t *screen_h, uint8_t *video_rotate)
172 {
173     if (lcdstruct.lcd_bus_type == LCD_BUS_I80)
174     {
175         if (lcdstruct.color_mode == LCD_MODE_666)
176         {
177             lcd hardware_init666();
178         }
179         else if (lcdstruct.color_mode == LCD_MODE_565)
180         {
181             lcd hardware_init565();
182         }
183         else
184         {
185             lcd hardware_init565();
186         }
187     }
188     else if (lcdstruct.lcd_bus_type == LCD_BUS_RGB)
189     {
190         lcd hardware_init565();
191     }
192     else if (lcdstruct.lcd_bus_type == LCD_BUS_MIPI)
193     {
194         lcd hardware_init565();
195     }
196     else if (lcdstruct.lcd_bus_type == LCD_BUS_SPI4){
197         lcd hardware_init565();
198     }
199
200     // 返回osd的w和h
201     get_osd_w_h(w, h, rotate);
202     get_screen_w_h(screen_w, screen_h, video_rotate);
203 }

```

2) app_lcd.c 文件下 lcd hardware_init565 函数中是否以添加 LCD_BUS_SPI4 相关配置。

```

205 // 主要硬件初始化,但由于存在rgb屏,还是会有部分寄存器需要配置
206 void lcd hardware_init565()
207 {
208     uint8 pixel_dot_num = 1;
209     struct lcdc_device *lcd_dev;
210     struct spi_device *spi_dev;
211
212     lcd_dev = (struct lcdc_device *)dev_get(HG_LCDC_DEVID);
213     spi_dev = (struct spi_device *)dev_get(HG_SPI0_DEVID);
214
215     uint8_t rst = MACRO_PIN(PIN_LCD_RESET);
216
217     if (rst != 255) --
218     if (lcdstruct.lcd_bus_type == LCD_BUS_RGB) --
219
220     // scale_to_lcd_config();
221     lcdc_init(lcd_dev);
222
223     if (lcdstruct.lcd_bus_type == LCD_BUS_I80) --
224     lcdc_set_color_mode(lcd_dev, lcdstruct.color_mode);
225     lcdc_set_bus_width(lcd_dev, lcdstruct.bus_width);
226     lcdc_set_interface(lcd_dev, lcdstruct.lcd_bus_type);
227     lcdc_set_colrarray(lcd_dev, lcdstruct.colrarray);
228     pixel_dot_num = lcdstruct.color_mode / lcdstruct.bus_width;
229     lcdc_set_lcd_vaild_size(lcd_dev, lcdstruct.screen_w + lcdstruct.hlw + lcdstruct.hbp + lcdstruct.hfp, lcdstruct.screen_h + lcdstruct.vlw + lcdstruct.vhp);
230     lcdc_set_lcd_visible_size(lcd_dev, lcdstruct.screen_w, lcdstruct.screen_h, pixel_dot_num);
231
232     if (lcdstruct.lcd_bus_type == LCD_BUS_I80) --
233     else if ((lcdstruct.lcd_bus_type == LCD_BUS_RGB) || (lcdstruct.lcd_bus_type == LCD_BUS_MIPI) || (lcdstruct.lcd_bus_type == LCD_BUS_QSPI)) --
234     lcdc_set_baudrate(lcd_dev, lcdstruct.pclk);
235     lcdc_set_bigendian(lcd_dev, 1);
236
237     if (lcdstruct.lcd_bus_type == LCD_BUS_I80) --
238     else if (lcdstruct.lcd_bus_type == LCD_BUS_MIPI)
239     {
240         mipi_dsi_init(lcdstruct.screen_w, lcdstruct.screen_h, lcdstruct.pclk, lcdstruct.vlw, lcdstruct.vbp, lcdstruct.vfp, lcdstruct.hlw, lcdstruct.hbp, lcdstruct.hfp);
241     }
242     else if (lcdstruct.lcd_bus_type == LCD_BUS_SPI4)
243     {
244         lcdc_spi_cs_lock_time(lcd_dev, 0x20, 0x20, 0x20);
245         lcdc_cmd_auto_send(lcd_dev, 0, 0);
246         lcdc_open(lcd_dev);
247         lcd_reg_table_init(lcd_dev, lcdstruct.lcd_bus_type, (uint8_t *) (lcdstruct.init_table));
248     }
249 }

```

lcd_spi_cs_lock_time 的三个参数作用: 一个是 cs 拉低后, 什么时候出数据; 一个是数据结束后, 什么时候拉高 cs; 一个 cs 间的最小 hold 间隔时间, 一般不需要调整。

1.6. 运行 lvgl benchmark 测试出图

1) app_fpv.c 打开 lvgl_init_msi 组件函数。

```
project > app > C app_fpv.c > ...
158 void hardware_init(uint8_t vcam)
299 #if LCD_EN
310 #else
311     struct hg_lv_mem_hooks hook = {
312         .zalloc = av_psram_zalloc,
313         .zalloc = av_psram_zalloc,
314         .zalloc = av_psram_zalloc,
315         .free = av_psram_free,
316     };
317 #endif
318     ha_lv_mem_register(&hook);
319     lvgl_init_msi(w, h, rotate);
320     //lcd_demo_thread(4);
321 #endif
322
323 #if ISP_TUNNING_EN
324     void isp_tunning_init();
325     isp_tunning_init();
326 #endif
327     user_hardware_config();
328 }
```

2) interface_mngnt_v3.c 文件中屏蔽 main_ui, 添加 lv_demo_benchmark。

```
sdk > app > interface_management > C interface_mngnt_v3.c
387 void lvgl_init_msi(uint16_t w,uint16_t h,uint8_t rotate)
414 #if 0
418     lv_style_set_shadow_color(&g_style, lv_color_make(0x00, 0x00, 0x00));
419     lv_style_set_border_color(&g_style, lv_color_make(0x00, 0x00, 0x00));
420     lv_style_set_outline_color(&g_style, lv_color_make(0x00, 0x00, 0x00));
421     lv_style_set_radius(&g_style, 0);
422 #endif
423
424     lv_demo_benchmark(1);
425
426     //main_ui(NULL);
427
428
429 #if 0
430     //lv_obj_add_style(lv_scr_act(),&g_style,0);
431     lv_obj_t *label = lv_label_create(lv_scr_act());
432     lv_obj_set_size(label, LV_PCT(100), LV_PCT(100));
433     lv_label_set_text(label, "");
434     lv_obj_align(label,LV_ALIGN_TOP_MID,0,0);
435     lv_timer_create(modify_time, 500, (void*)label);
436 #endif
437     os_task_create("gui_thread", lvgl_run_msi, NULL, OS_TASK_PRIORITY_NORMAL, 0, NULL, 4096);
438
439 }
```

3) 编译运行, 下载到板子上, 查看点亮情况。

Note: 如果有字体报错提醒, 可以在 lv_config.h 中打开对应字号字体。

2. MIPI-DSI 屏幕驱动配置讲解

2.1. 简要说明

相比之前 81x 芯片 82x lcdc 模块进行了升级，可以支持 mipi 屏幕。配置一个 mipi-dsi 屏幕大致步骤，配置 io，lcdc 参数，替换 mipi 屏幕驱动，编译调试出图，分享一些 mipi 调试技巧

Note: 以 release sdk txw82x_fpv_2.7.07-37092_beta 版本为基础介绍。

2.2.DSI IO 配置

MIPI 屏幕常见控制引脚有，DSI_CLKN/P，DSI_D0N/P，DSI_D1N/P 用于命令数据通讯，以及 RES，BACK_LIGHT，复位和背光控制引脚。

```
/*mipi dsi io 配置，引脚可以按组映射，N P 引脚可以互换*/

#define PIN_MIPI_DSI_CLKN PD_7//default PD_2

#define PIN_MIPI_DSI_CLKP PD_6//default PD_3

#define PIN_MIPI_DSI_D0N PD_5//default PD_4

#define PIN_MIPI_DSI_D0P PD_4//default PD_5

#define PIN_MIPI_DSI_D1N PD_3//default PD_6

#define PIN_MIPI_DSI_D1P PD_2//default PD_7

/**4 lane 只有 96pin 封装支持 不常用 ***/

#define PIN_MIPI_DSI_D2N 255//default PD_8

#define PIN_MIPI_DSI_D2P 255//default PD_9

#define PIN_MIPI_DSI_D3N 255//default PD_10

#define PIN_MIPI_DSI_D3P 255//default PD_11

#define PIN_LCD_BKLIGHT PB_8 //背光控制引脚 需要用户自己定义
```

```
#define PIN_LCD_RESET    PB_9 // lcd 复位控制引脚，硬件电路也可以直接硬件控制
```

Note: 如果 project_conf.h 中开启 **PIN_FROM_PARAM**，以上 PIN 脚 IO 定义是通过脚本和 config.cfg 配置去实现的。如果遇到 4lane 的屏幕接 82x_2lane 的芯片需要确定屏幕硬件或者软件以将屏幕设置成 2lane 模式。

2.3.MIPI-DSI 接口 LCDC 参数配置

82x 芯片的 lcdc 模块配置成 mipi-dsi 接口，在 lcdstruct 结构体中，如果你要加一份新的驱动芯片，可以模仿 st7701s_mipi.c 的配置另 copy 一份.c 文件。重点关注以下几点：

- 1) 总线相关配置默认跟 st7701s_mipi 一致：
 - lcd_bus_type (总线类型)，bus_width (总线宽度)，color_mode (颜色模式)
- 2) 旋转角度配置：
 - osd_scan_mode (osd 层 lvgl 的旋转角度)，按需配置。
 - scan_mode (video 层的旋转角度特殊，谨慎使用) 按需配置。
- 3) 总线时钟速率：
 - pclk (mipi 的像素时钟)，根据 lcd 驱动芯片要求去适配。

目前 7701 配置的 24Mhz，那么 2lane 高速模式下单 lane 速率就要是 $24 \times 3 \times 8 / 2 = 288\text{Mhz}$

帧率就是 像素点+porch 约等于 51FPS，帧率够用就好，如果一味的提升会增加功耗和 psram 的带宽压力。

- 4) lane_num 数量配置：根据实际屏幕需求设置 1lane 还是 2lane。
- 5) lcd 有效显示尺寸设置：
 - screen_w/h：需要按你实际驱动芯片的显存宽高来设置。
 - video_w/h 和 osd_w/h：分别是 video 和 osd 层有效显示宽高，按实际显示范围设置。
 - video_x/y 和 osd_x/y：分别是 video 和 osd 层偏移，通过计算得来。
- 6) porch 设置：

也就是行场消隐，根据屏厂要求填写，屏幕不同 porch 设置不同。

7) init_table 是 lcd 寄存器初始化表(找屏厂索取)记得 END 结尾, frame_table 暂时可以不用。

```
sd&gt; lib > bus > spi > lcd > C st7701s_mipi.c > lcdstruct
3  #if 1//LCD_ST7701S_MIPI_EN
584
585  lcddev_t lcdstruct = {
586      .name = "st7701s_mipi",
587      .lcd_bus_type = LCD_BUS_MIPI,
588      .bus_width = LCD_BUS_WIDTH_24,
589      .color_mode = LCD_MODE_888,
590      //red_width = LCD_BUS_WIDTH_6,
591      //green_width = LCD_BUS_WIDTH_6,
592      //blue_width = LCD_BUS_WIDTH_6,
593      .osd_scan_mode = LCD_ROTATE_0,//LCD_ROTATE_270
594      .scan_mode = LCD_ROTATE_270,// LCD_ROTATE_90,//rotate 90
595      .te_mode = 0xff,//te mode, 0xff:disable
596      .colrarray = 0,//0:_RGB_ 1:_RBG_,2:_GBR_,3:_GRB_,4:_BRG_,5:_BGR_
597      //f(wr) = source_clk/div/2
598      //f(wr) >= screen_w * screen_h * clk_per_pixel * 60
599      .pclk = DEFAULT_SYS_CLK/(DEFAULT_SYS_CLK/24000000),
600      .even_order = 0,
601      .odd_order = 0,
602      .lcd_data_mode = (0<<31) //data inversion mode
603                      (2<<24) //data compress mode
604                      (1<<20) //fifo mode
605                      (0<<17) //output cycle 2 shift direction
606                      (0<<12) //output cycle 2 shift bit
607                      (0<<11) //output cycle 1 shift direction
608                      (0<<6)  //output cycle 1 shift bit
609                      (0<<5)  //output cycle 0 shift direction
610                      (8<<0)  //output cycle 0 shift bit
611      .screen_w = 480,
612      .screen_h = 800,
613      .video_x = 0,
614      .video_y = 0,
615      .video_w = 480,//480,
616      .video_h = 800,//800,
617      .osd_x = 0,
618      .osd_y = 0,
619      .osd_w = 480, // 0 : value will set to video_w , use for 4:3 LCD +16:9 sensor show UPDOWN BLACK
620      .osd_h = 800, // 0 : value will set to video_h , use for 4:3 LCD +16:9 sensor show UPDOWN BLACK
621      .init_table = st7701s_register_init_tab,
622      .frame_table = NULL,
623      .clk_per_pixel = 2,
624      .lane_num = 2,
625      .pclk_inv = 1,
626
627      .vlw = 2,
628      .vbp = 2,
629      .vfp = 10,
630
631      .hlw = 2,
632      .hbp = 8,
633      .hfp = 96,
634
635
636      .de_inv = 0,
637      .hs_inv = 0,
638      .vs_inv = 0,
639
640
641      .de_en = 1,
642      .vs_en = 1,
643      .hs_en = 1,
```

Note: mipi 需要注意一下 DSI 单 lane 速率上限, 如果所需于不高于 480M 就选 DSI_MODULE_CLK480, 高于 480M 就选 DSI_MODULE_CLK_960M, 在 mipi_dsi_driver.c 中配置

2.4.MIPI-DSI 屏幕调试技巧

- **Step1:** 打开 turnaround 模式，查看发送的初始化命令是否会卡住，是否完全发送。

```
sdk > lib > lcd > C mipi_dsi_driver.c > mipi_dsi_init(uint32, uint32, uint32, uint8, uint8, uint8, uint8, uint8, uint8, uint8)
400 void mipi_dsi_init(uint32 w,uint32 h,uint32 dclk,uint8 vsa,uint8 vbp,uint8 vfp,uint8 hsa,uint8 hbp,uint8
436 mipi_dsi_gen_vcid_cfg(dsi_dev,GENERIC_VC_ID,1,1);
437
438 //mipi_dsi_cmd_mode_cfg(dsi_dev,1,1,0,0,0,0,0,0,0,0,0,0,0,0);
439 //mipi_dsi_cmd_mode_cfg(dsi_dev,1,0,1,1,1,1,1,1,1,1,1,1,1,1);
440 mipi_dsi_cmd_mode_cfg(dsi_dev,0,1,1,1,1,1,1,1,1,1,1,1,1,1); //turnaround mode
441
442 mipi_dsi_dpi_vcid(dsi_dev,0);
443 if(colortype == LCD_MODE_565){
444 mipi_dsi_color_cfg(dsi_dev,0,0); //5:24BIT 0:16BIT
```

- **Step2:** 用逻辑分析仪（逻辑电平 1.2V 或者 1.5V）抓取一下 LowPowerMode 下 dat0lane 发送的 cmd 是否有回应。
- **Step3:** 在 0x29 打开显示命令前，发送 0x23 看看是否像素点全亮，或者让 lcd 驱动芯片进入自测模式。需要在 dsi 切换高速模式前 while(1)住。

```
sdk > lib > bus > spi > lcd > C st7701s_mipi.c > st7701s_register_init_tab
3 #if LCD_ST7701S_MIPI_EN
...
558 CMD(0x3A),
559 DAT(0x77),
560 #if 0 //test pattern test mode
561 CMD(0xFF),
562 DAT(0x77),
563 DAT(0x01),
564 DAT(0x00),
565 DAT(0x00),
566 DAT(0x12),
567
568 CMD(0xD1),
569 DAT(0x81),
570
571 CMD(0xD2),
572 DAT(0x06),
573 #endif
574 CMD(0x29),
575
576 // DLY(120),
577 //~ CMD(0x23), display white
578 // DLY(120),
579 #endif

sdk > lib > lcd > C mipi_dsi_driver.c > mipi_dsi_init(uint32, uint32, uint32, uint8, uint8, uint8, uint8, uint8, uint8, ui
400 void mipi_dsi_init(uint32 w,uint32 h,uint32 dclk,uint8 vsa,uint8 vbp,uint8 vfp,uint8 hsa,uint8 hbp,uint8 ui
503
504 while(1);... stop dsi switch mode
505 mipi_dsi_select_mode(dsi_dev,0); // switch dsi high speed mode
506
507
508 }
```

- **Step4:** 读取 lcd 寄存器状态，根据芯片手册查看对应寄存器状态是否正确。

4. LCD 屏幕抖动

4.1. dclk 补全设置

留心这里设置，如果没有就在 lcd_hardware_init565 添加打开，这里是处理帧间隔停 clk 抖动的屏幕，rgb/mipi 屏幕均建议开启。

```
if((lcd_struct.lcd_bus_type == LCD_BUS_RGB)
    lcdc_clock_alway_on(lcd_dev,1);
} ? end lcd_hardware_init565 ?
```

4.2. 缩短帧间隔设置

如果 video 旋转角度是 0 或者 180，那么推荐尝试改小 linebuf 行数减少等待时间，目前默认是 64 行，可以修改成 32，16，8。这里是处理某些对帧间隔要求比较严格导致抖动的屏幕。

```
#define LCD_ROTATE_LINE 16//8//16//32//64
```

4.3. 修改 dma burst

修改 device.c 中 dma 设置，修改 DMA2AHB_BURST_CH_ROTATE_IN_RD 对 psram 占用效率。其余改小 DMA2AHB_BURST_CH_ROTATE_IN_RD 改大。如果 video 旋转的角度只是 0 度或者 180 度，DMA2AHB_BURST_CH_ROTATE_IN_RD 可以设置到最大，如果旋转角度是 90 度或者 270 度遵循 pingpang 原则，burst 设置 32，linebuf 行数就是 32x2=64。

```
void device_burst_set(void)
{
    *(unsigned int *) 0x400201d0 = 0xffffffff;
    *(unsigned int *) 0x400201d4 = 0xffffffff;

    // SYSCTRL->AHB2AHB_FIFO_CTRL = 0x1FFFFFF7;
    #if 1
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_DUAL_ORG0_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_DUAL_ORG1_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_DUAL_ORG_RD, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_ROTATE_IN_RD, DMA2AHB_BURST_SIZE_64); //DMA2AHB_BURST_SIZE_32

        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_M2M0_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_M2M0_RD, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_M2M1_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_M2M1_RD, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_SDHOST1_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_M2M2_RD, DMA2AHB_BURST_SIZE_32);

        //ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_SCALE3_Y_WR, DMA2AHB_BURST_SIZE_256);
        //ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_SCALE3_U_WR, DMA2AHB_BURST_SIZE_256);
        //ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_SCALE3_V_WR, DMA2AHB_BURST_SIZE_256);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_VPP_Y_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_VPP_U_WR, DMA2AHB_BURST_SIZE_32);
        ll_sysctrl_dma2ahb_burst_set(DMA2AHB_BURST_CH_GMAC_RX_WR, DMA2AHB_BURST_SIZE_32);
    #endif
}
```

4.4.添加 osd fifo

在 app_lcd.c 中 lcd_driver_init 函数中添加下图代码段, 对 osd 抖动有一定优化, fifo size 可以适当加大

```
00531:     lcdc_set_p0_rotate_y_src_addr(lcd_dev, (uint32) 0x28000000);
00532:     lcdc_set_p0_rotate_u_src_addr(lcd_dev, (uint32) 0x28000000);
00533:     lcdc_set_p0_rotate_v_src_addr(lcd_dev, (uint32) 0x28000000);
00534:     #if 1
00535:         uint32_t *osd_sram_fifo;
00536:         lcdc_osd_sram_fifo_enable(lcd_dev,1);
00537:         osd_sram_fifo = os_malloc(512);
00538:         lcdc_osd_sram_fifo_addr(lcd_dev,osd_sram_fifo,osd_sram_fifo+(512/4-1));
00539:     #endif
00540:     lcdc_set_osd_start_location(lcd_dev, lcdstruct.osd_x, lcdstruct.osd_y);
00541:     lcdc_set_osd_size(lcd_dev, lcdstruct.osd_w, lcdstruct.osd_h);
00542:     lcdc_set_osd_format(lcd_dev, OSD_RGB_565);
```

4.5.OTA 出现抖动

空口升级或者写 flash 保存参数的时候, 有概率会 holdcpu, 导致屏幕抖动, 建议开启 lcd autokick

4.6.大屏 video+osd 频繁解码出现抖动的情况

以上处理完还出现抖动的情况, 且会频繁出现 lch 打印, 建议把 osd_fifo 和 video line_buf 放到 sram 最后面, 减少内存竞争。

5. MCU 和 SPI 屏幕利用外部中断做 TE 同步

5.1. 简要说明

带显存的屏幕都会面临撕裂 TE 同步问题, 目前 sdk 默认是没有处理 TE 同步, 可以参考下面添加 82x TE 同步, 通过外部中断的方式。

- **Step1:** 在 app_lcd.c 中设置 TE 上升沿触发外部中断 lcd_extern_te_isr。

```

00500:     lcdc_screen_yuv_addr(lcd_dev, video_psram_screen, video_psram_screen + SCALE_CONFIG_W * SCALE_HIGH, video_psram_scr
00501:     lcdc_request_irq(lcd_dev, SCREEN_DONE_IRQ, (lcdc_irq_hdl) &lcd_screen_finish, (uint32) lcd_s);
00502: #endif
00503: // lcdc_request_irq(lcd_dev, OSD_EN_IRQ, (lcdc_irq_hdl) &lcd_osd_isr1_msi, (uint32) lcd_s);
00504: lcdc_request_irq(lcd_dev, TIMEOUT_IRQ, (lcdc_irq_hdl) &lcd_timeout1, (uint32) lcd_s);
00505: if (MACRO_PIN(LCD_TE) != 255)
00506: {
00507:     os_printf("#### LCD_TE _INIT \n\r");
00508:     #if 1
00509:     {
00510:         uint8_t te_pin = MACRO_PIN(LCD_TE);
00511:
00512:         if (te_pin != 255)
00513:         {
00514:             gpio_iomap_input(te_pin, GPIO_IOMAP_INPUT);
00515:             gpio_ioctl(te_pin, GPIO_DEBUNCE, 1, 0);
00516:             gpio_request_pin_irq(te_pin, lcd_extern_te_isr1, NULL, GPIO_IRQ_EVENT_RISE);
00517:         }
00518:     }
00519:     #else
00520:     lcdc_te_edge_cfg(lcd_dev, 0);
00521:     lcdc_request_irq(lcd_dev, LCD_TE_IRQ, (lcdc_irq_hdl) &lcd_te_isr1, (uint32) lcd_s);
00522:     #endif
00523: }
00524: lcdc_open(lcd_dev);
00525:
00526: #if 0
00527: //extern void lcd_thread(void *d);
00528: //lcd_s->thread_hdl = os_task_create("lcd_thread", lcd_thread2, lcd_s, OS_TASK_PRIORITY_NORMAL, 0, NULL, 1024);
00529: #else
00530: extern int32 lcd_msi_work(struct os_work * work);
00531:

```

- **Step2:** 在 app_lcd.c 中添加外部中处理函数 lcd_extern_te_isr1, 每次 TE 信号触发 IO 部中断时数据发送同步。

```

// 暂时没有作处理
static void lcd_te_isr1(uint32 irq_flag, uint32 irq_data, uint32 param1)
{
    os_printf("%s:%d\n", __FUNCTION__, __LINE__);
    struct app_lcd_s *lcd_s = (struct app_lcd_s *) irq_data;
    struct lcdc_device *lcd_dev;
    lcd_dev = lcd_s->lcd_dev;
    os_printf("-----te-----\n\r");
}

volatile uint8_t lcd_frame_flash=0;

static void lcd_extern_te_isr1(int32 id, enum gpio_irq_event event, uint32 param1, uint32 param2)
{
    struct lcdc_device *lcd_dev;
    lcd_dev = (struct lcdc_device *)dev_get(HG_LCDC_DEVICE);
    if(lcd_frame_flash)
    {
        lcdc_set_timeout_info(lcd_dev, 1, 3);
        lcdc_set_start_run(lcd_dev);
        lcd_frame_flash=0;
    }
    //os_printf("T");
}

```

- **Step3:** 在 lcd_msi.c 文件中的 lcd_msi_irq_callback 判断是否使能 TE, 做选择直接 kick lcd 还是在同步中断中 kick lcd。

```

if(MACRO_PIN(LCD_TE) != 255)
{
    lcd_frame_flash=1;
}
else
{
    lcdc_set_timeout_info(lcd_s->lcd_dev, 1, 3);
    #ifndef YIHEDA_BOARD
    lcdc_set_start_run(lcd_s->lcd_dev);
    #endif
}

```

6. MCU 和 SPI 屏幕，增加帧间命令操作

某些特殊不通用的 LCD 控制器（格科微的 GC9309NA 等一系列芯片，矽创的 st77926），如果需要在数据帧与帧之间添加命令的情况可以查看下面流程。

- **Step1:** 屏幕驱动中，填写需要发送的帧间命令，譬如需要添加 2A, 2B, 2C 命令。

```
15  uint8_t gc9307_firt_send[][2]={
16      CMD( 0x2A),
17      DAT( 0x00),
18      DAT( 0x00),
19      DAT( 0x00),
20      //DAT( 0x7F),
21      DAT( 0x83),
22
23      CMD( 0x2B),
24      DAT( 0x00),
25      DAT( 0x00),
26      DAT( 0x00),
27      //DAT( 0x9F),
28      DAT( 0xA1),
29
30      CMD( 0x2C),
31      END
32  };
```

- **Step2:** 屏幕驱动中，把帧间命令 table 赋值给 frame_table 指针变量。

```
302  .init_table = gc9307_register_init_tab,
303  .frame_table = gc9307_firt_send,
304  .clk_per_pixel = 2,
```

- **Step3:** 声明定义帧间命令发送函数。

```
68  void mcu_spi_lcd_framecmd_send(void)
69  {
70      #if defined(MCUSPI_LCD_FRAMECMD_EN)
71          struct lcdc_device *lcd_dev;
72          lcd_dev = (struct lcdc_device *) dev_get(HG_LCDC_DEVID);
73
74          if ((lcdstruct.lcd_bus_type == LCD_BUS_SPI4) || (lcdstruct.lcd_bus_type == LCD_BUS_I80))
75          {
76              *(volatile uint32 *)0x40007008 |= BIT(5); //before 43316-sdk-version ,must add
77
78              lcd_reg_table_init(lcd_dev, lcdstruct.lcd_bus_type, (uint8_t *) (lcdstruct.frame_table));
79          }
80      #endif
81  }
82
```

- **Step4:** 添加帧间发送函数到指定位置，分两种情况带 TE 和不带 TE 同步。

- 1) 带 TE 控制的，需要在 TE 中断中添加发送，这里已经默认按点 5 指导添加了 TE 处理。

```
87  volatile uint8_t lcd_frame_flash=0;
88
89  static void lcd_extern_te_isr(int32 id, enum gpio_irq_event event, uint32 param1, uint32 param2)
90  {
91      struct lcdc_device *lcd_dev;
92      lcd_dev = (struct lcdc_device *)dev_get(HG_LCDC_DEVID);
93      if(lcd_frame_flash)
94      {
95          lcdc_set_timeout_info(lcd_dev,1,3);
96          mcu_spi_lcd_framecmd_send();
97          lcdc_set_start_run(lcd_dev);
98          lcd_frame_flash=0;
99      }
100      //os_printf("T");
101  }
102
```

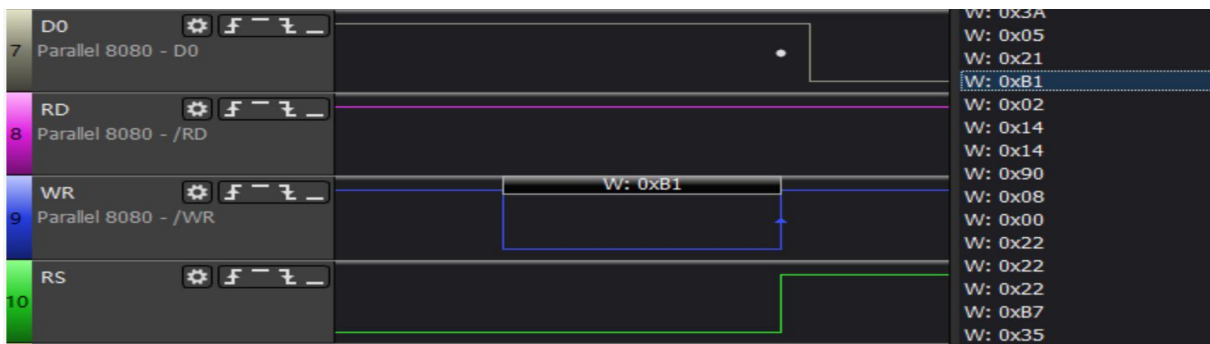
- 2) 不带 TE 控制的, 也就是 LCD_TE 设置 255 的情况。需要在 lcd_msi.c->lcd_msi_irq_call 函数 lcdc_set_start_run 调用前添加

```
427     if(MACRO_PIN(LCD_TE) != 255)
428     {
429         lcd_frame_flash=1;
430     }
431     else
432     {
433         lcdc_set_timeout_info(lcd_s->lcd_dev, 1, 3);
434         #if !defined(YIHEDA_BOARD) && !defined(GUQIAO_BOARD)
435         mcu_spi_lcd_framecmd_send();
436         lcdc_set_start_run(lcd_s->lcd_dev);
437         #endif
438     }
439     lcd_s->start_time = os_jiffies();
440 }
441
442
443 }
```

Note: 核心目的是在 kick lcd 发送显存前, 把需要的帧间命令发送出去。

7. MCU 屏幕, WR 信号跳变不及时导致发送出错

SDK-43316 之前的版本会存在这个问题, WR 跳变不及时导致数据通讯出错需要添加如下:



需要在 app_lcd.c->lcd_hardware_init565 函数 lcdc_open 之后添加 lcdc_clk_gather_select 设置修改出数据和锁存数据时机。

```
311     // scale to lcd_config();
312     lcdc_init(lcd_dev);
313     if ((lcdstruct.lcd_bus_type == LCD_BUS_I80) || (lcdstruct.lcd_bus_type == LCD_BUS_SPI4))
314     {
315         lcdc_open(lcd_dev);
316     }
317     lcdc_set_color_mode(lcd_dev, lcdstruct.color_mode);
318     lcdc_set_bus_width(lcd_dev, lcdstruct.bus_width);
319     lcdc_set_interface(lcd_dev, lcdstruct.lcd_bus_type);
320     lcdc_set_colrarray(lcd_dev, lcdstruct.colrarray);
321     pixel_dot_num = lcdstruct.color_mode / lcdstruct.bus_width;
322     lcdc_set_lcd_vaild_size(lcd_dev, lcdstruct.screen_w + lcdstruct.hlw + lcdstruct.hbp + lcdstruct.hfp, lcdstruct.screen_h, pixel_dot_num);
323     lcdc_set_lcd_visible_size(lcd_dev, lcdstruct.screen_w, lcdstruct.screen_h, pixel_dot_num);
324     if (lcdstruct.lcd_bus_type == LCD_BUS_I80)
325     {
326         lcdc_clk_gather_select(lcd_dev, 1);
327     }
328     lcdc_mcu_signal_config(lcd_dev, lcdstruct.signal_config.value);
329 }
```

8. MCU 屏幕, ID 读取操作设置

需要通过读取接口获取 MCU 屏幕状态寄存器或者获取屏幕 ID 操作的可以参考如下修改。

- **Step1:** 接口设置，在初始化前添加 `lcdc_reg_read_data` 读取数据，其中传入参数介绍。
 - `pt8[0]`->命令长度，0xda 一个字节
 - `pt8[1]`->读取数据长度，下图填的是 4 字节
 - `pt8[2]`->读取的寄存器地址
 - `pt8[3]`->读取到的数据存放位置

```

340     lcdc_set_bigendian(lcd_dev, 1);
341     if (lcdstruct.lcd_bus_type == LCD_BUS_I80)
342     {
343         //     pt8 = (uint8_t *)rbuf;
344         //     pt8[0] = 1;
345         //     pt8[1] = 4;
346         //     pt8[2] = 0xda;
347         //     lcdc_reg_read_data(lcd_dev, 1, 4, rbuf);
348         lcd_reg_table_init(lcd_dev, lcdstruct.lcd_bus_type, (uint8_t *) (lcdstruct.init_table));
349     }
350     else if (lcdstruct.lcd_bus_type == LCD_BUS_MIPi)
351     {
352         mipi_dsi_init(lcdstruct.screen_w, lcdstruct.screen_h, lcdstruct.pclk, lcdstruct.vlw, lcdstruct.vbp, lcds
353             lcdstruct.color_mode);
354     }
355     else if (lcdstruct.lcd_bus_type == LCD_BUS_SPI4)

```

- **Step2:** Pin 脚设置，确保 RD 引脚配置不是 255。其次确定一下

pin_function.c->lcdc_pin_func

要把 LCD_D0~D7 设置为 inout 模式, 尤其 sdk-43316 之前版本需要注意。

```

1068 static int lcdc_pin_func(int dev_id, int request)
1069 {
1070     int ret = RET_OK;
1071     switch (dev_id) {
1072         case HG_LCDC_DEVID:
1073             gpio_iomap_output(MACRO_PIN(LCD_VS_CS), GPIO_IOMAP_OUT_LCD_VSYNC_OR_CS0);
1074             gpio_iomap_output(MACRO_PIN(LCD_HS_DC), GPIO_IOMAP_OUT_LCD_HSYNC_OR_DC);
1075             gpio_iomap_output(MACRO_PIN(LCD_DE_ERD), GPIO_IOMAP_OUT_LCD_DE_OR_ERD1);
1076             #if (WIFI_FEM_CHIP == LMAC_FEM_NONE)
1077                 gpio_iomap_output(MACRO_PIN(LCD_DOTCLK_RWR), GPIO_IOMAP_OUT_LCD_DOTCLK_OR_RWR0);
1078             #else
1079                 gpio_iomap_output(MACRO_PIN(LCD_DOTCLK_RWR), GPIO_IOMAP_OUT_LCD_DOTCLK_OR_RWR1);
1080             #endif
1081         #if 1
1082             gpio_iomap_input(MACRO_PIN(LCD_TE), GPIO_IOMAP_IN_PORT_WKUP_IN3_LCD_TE_M0_10);
1083             gpio_iomap_input(MACRO_PIN(LCD_D0), GPIO_IOMAP_IN_LCD_D0_IN_M1_19, GPIO_IOMAP_OUT_LCD_DATA_0_0);
1084             gpio_iomap_input(MACRO_PIN(LCD_D1), GPIO_IOMAP_IN_LCD_D1_IN_M1_20, GPIO_IOMAP_OUT_LCD_DATA_0_1);
1085             gpio_iomap_input(MACRO_PIN(LCD_D2), GPIO_IOMAP_IN_LCD_D2_IN_M1_21, GPIO_IOMAP_OUT_LCD_DATA_0_2);
1086             gpio_iomap_input(MACRO_PIN(LCD_D3), GPIO_IOMAP_IN_STMR0_CAP_IN_LCD_D3_IN_M1_22_EPM1_SYNC_IO_M3_6, GPIO_IOMAP_OUT_LCD_DATA_0_3);
1087             gpio_iomap_input(MACRO_PIN(LCD_D4), GPIO_IOMAP_IN_STMR1_CAP_IN_LCD_D4_IN_M1_23, GPIO_IOMAP_OUT_LCD_DATA_0_4);
1088             gpio_iomap_input(MACRO_PIN(LCD_D5), GPIO_IOMAP_IN_STMR2_CAP_IN_LCD_D5_IN_M1_24, GPIO_IOMAP_OUT_LCD_DATA_0_5);
1089             gpio_iomap_input(MACRO_PIN(LCD_D6), GPIO_IOMAP_IN_STMR3_CAP_IN_LCD_D6_IN_M1_25, GPIO_IOMAP_OUT_LCD_DATA_0_6);
1090             gpio_iomap_input(MACRO_PIN(LCD_D7), GPIO_IOMAP_IN_PORT_WKUP_IN1_LCD_D7_IN_M1_26, GPIO_IOMAP_OUT_LCD_DATA_0_7);
1091             gpio_iomap_input(MACRO_PIN(LCD_D8), GPIO_IOMAP_IN_PORT_WKUP_IN2_LCD_D8_IN_M1_27, GPIO_IOMAP_OUT_LCD_DATA_0_8);
1092             gpio_iomap_input(MACRO_PIN(LCD_D9), GPIO_IOMAP_IN_UART1_IN_LCD_D9_IN_M1_28, GPIO_IOMAP_OUT_LCD_DATA_0_9);
1093             gpio_iomap_input(MACRO_PIN(LCD_D10), GPIO_IOMAP_IN_UART1_CTS_DE_IN_LCD_D10_IN_M1_29, GPIO_IOMAP_OUT_LCD_DATA_0_10);
1094             gpio_iomap_input(MACRO_PIN(LCD_D11), GPIO_IOMAP_IN_SPI1_NSS_IN_LCD_D11_IN_M1_30, GPIO_IOMAP_OUT_LCD_DATA_0_11);
1095             gpio_iomap_input(MACRO_PIN(LCD_D12), GPIO_IOMAP_IN_SPI1_IO1_IN_LCD_D12_IN_M1_31, GPIO_IOMAP_OUT_LCD_DATA_0_12);
1096             gpio_iomap_input(MACRO_PIN(LCD_D13), GPIO_IOMAP_IN_SPI2_NSS_IN_LCD_D13_IN_M2_0, GPIO_IOMAP_OUT_LCD_DATA_0_13);
1097             gpio_iomap_input(MACRO_PIN(LCD_D14), GPIO_IOMAP_IN_SPI2_IO1_IN_LCD_D14_IN_M2_1, GPIO_IOMAP_OUT_LCD_DATA_0_14);
1098             gpio_iomap_input(MACRO_PIN(LCD_D15), GPIO_IOMAP_IN_SPI2_IO2_IN_LCD_D15_IN_M2_2, GPIO_IOMAP_OUT_LCD_DATA_0_15);
1099             gpio_iomap_input(MACRO_PIN(LCD_D16), GPIO_IOMAP_IN_SPI2_IO3_IN_LCD_D16_IN_M2_3, GPIO_IOMAP_OUT_LCD_DATA_0_16);
1100             gpio_iomap_input(MACRO_PIN(LCD_D17), GPIO_IOMAP_IN_STMR4_CAP_IN_LCD_D17_IN_M2_4, GPIO_IOMAP_OUT_LCD_DATA_0_17);
1101             gpio_iomap_input(MACRO_PIN(LCD_D18), GPIO_IOMAP_IN_STMR5_CAP_IN_LCD_D18_IN_M2_5, GPIO_IOMAP_OUT_LCD_DATA_0_18);
1102             gpio_iomap_input(MACRO_PIN(LCD_D19), GPIO_IOMAP_IN_IIS0_MCLK_IN_LCD_D19_IN_M2_6, GPIO_IOMAP_OUT_LCD_DATA_0_19);
1103             gpio_iomap_input(MACRO_PIN(LCD_D20), GPIO_IOMAP_IN_IIS0_WSClk_IN_LCD_D20_IN_M2_7, GPIO_IOMAP_OUT_LCD_DATA_0_20);
1104             gpio_iomap_input(MACRO_PIN(LCD_D21), GPIO_IOMAP_IN_IIS0_BCLK_IN_LCD_D21_IN_M2_8, GPIO_IOMAP_OUT_LCD_DATA_0_21);
1105             gpio_iomap_input(MACRO_PIN(LCD_D22), GPIO_IOMAP_IN_IIS1_MCLK_IN_UART5_IN_LCD_D22_IN_M2_9, GPIO_IOMAP_OUT_LCD_DATA_0_22);
1106             gpio_iomap_input(MACRO_PIN(LCD_D23), GPIO_IOMAP_IN_IIS1_WSClk_IN_LCD_D23_IN_M2_10, GPIO_IOMAP_OUT_LCD_DATA_0_23);
1107         #endif
1108     }
1109     return ret;
1110 }

```

9. LCD 录屏模块，检查显存操作

82x 芯片增加了录屏模块，除了可以做录屏应用，也可以方便我们排查问题，用于区分是应用层画图绘制异常，还是屏参驱动设置异常或者是屏幕本身异常。

只需要在 app_lcd.c 打开宏定义使能。

```
app_lcd.c
16  #ifdef LCD_EN
17
18  #define LCD_ROTATE_LINE 64
19
20  #define OSD_EN 1
21  #define LCD_SCREEN_RECORD_EN 0 // 录屏功能
22
23  struct app_lcd_s lcd_msg_s;
24
```

申请空间，注册中断后，每次 screen_done 中断产生都会把 yuv420p 格式的显存数据保存到 psram 中，也就是下图中的 video_psram_screen 指向的空间。

```
488
489  lcdc_request_irq(lcd_dev, LCD_DONE_IRQ, (lcdc_irq_hdl) &lcd_sqrualbuf_done1, (uint32) lcd_s);
490  #if LCD_SCREEN_RECORD_EN
491  uint8_t *video_psram_screen = (uint8_t *) av_psram_zalloc(SCALE_CONFIG_W * SCALE_HIGH + SCALE_CONFIG_W * SCALE_HIGH / 2);
492  os_printf("LCD_SCREEN_RECORD_EN psram addr: %x\n", video_psram_screen);
493  lcdc_screen_yuv_addr(lcd_dev, video_psram_screen, video_psram_screen + SCALE_CONFIG_W * SCALE_HIGH, video_psram_screen +
494  lcdc_request_irq(lcd_dev, SCREEN_DONE_IRQ, (lcdc_irq_hdl) &lcd_screen_finish, (uint32) lcd_s);
495  #endif
```

Note: 可以仿真 dump 出这段数据，之后用 ffmpeg 命令查看图像，dump 操作可以查看 81x lcd_faq 文档。

10. DITHER 模块使用介绍

82x 系列芯片支持 dither 功能，当然有些 lcd 屏幕也是支持 dither 功能，如果 lcd 屏幕不支持，可以尝试打开 82x 的 dither 功能，针对 rgb565 的图像做一下滤波处理，对于某些色彩较为单调的场景，如白色墙体，天空这类的，能减少过渡断层显示效果会有所提升。

打开方式如下图片设置：

首先设置 lcdc_dither_linebuf，3 行就够了。其次使能 lcdc_dither_en 就可以开启 dither 功能

```

app_lcd.c
uint8_t *dither_linebuf1;

// 芯片lcd的驱动初始化
void lcd_driver_init(const char *osd_encode_name, const char *lcd_osd_name, const char *lcd_video_p0, const char *lcd_video_p1)
{
    #if 1
    #endif
    #if 1
    #endif
    // lcdc_osd_enc_en(lcd_dev, 0);
    // LCD_SCREEN_RECORD_EN
    #endif

    #if 1 //dither
        dither_linebuf1 = malloc(lcdstruct.screen_w*3);
        lcdc_dither_linebuf(lcd_dev, (uint32)dither_linebuf1);
        lcdc_dither_en(lcd_dev, 1);
    #endif

    lcdc_open(lcd_dev);

    #if defined(YIHEDA_BOARD) || defined(YUAN_HENG_BOARD) || defined(GUQIAO_BOARD)
        lcdc_video_enable_auto_ks(lcd_dev, 1);
    #endif

    #if 0
        //extern void lcd_thread(void *d);
        //lcd_s->thread_hdl = os_task_create("lcd_thread", lcd_thread2, lcd_s, OS_TASK_PRIORITY_NORMAL, 0, NULL, 1024);
    #else
        extern int32 lcd_msi_work(struct os_work * work);
        extern void lcd_msi_irq_callback(void *data);
        lcd_s->app_lcd_cb = lcd_msi_irq_callback;
        OS_WORK_INIT(&lcd_s->work, lcd_msi_work, 0);
        os_run_work_delay(&lcd_s->work, 1);
    #endif
}

```

Note: 开启会占用 3 行大小的 sram，宽度越宽所需 sram 越多。目前芯片采用的是空间抖动的方式，使之周围像素看起来更平滑。

11. SCALE2 SCALE3 缩放模块使用介绍

需要关注以下三个函数接口：

- scale_set_in_out_size // 设置输入大小和输出大小
- scale_set_step // 设置实际缩放大小
- scale_set_start_addr // 设置相对于输入数据源的偏移

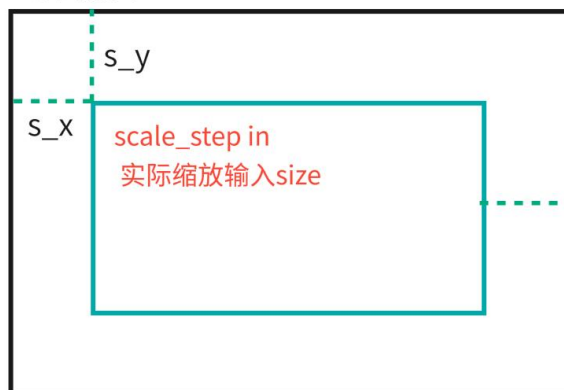
下图是常见的中心缩放显示功能举例，通过 at 命令改变 atcmd_enlarge，从而改变 scale_set_step 输入 size，在通过 scale_set_start_addr 设置坐标，atcmd_enlarge 范围假如 10~40。scale_set out 和 sscale_step out 固定时，10 到 40 变化会达到**中心拉近放大效果**。

```

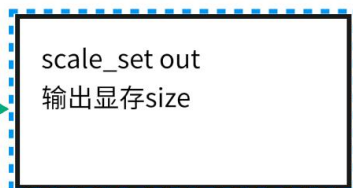
scale3_normal_msi.c
205  extern uint8_t atcmd_enlarge;
206
207  static int32_t scale3_stream_done(uint32_t irq_flag, uint32_t irq_data, uint32_t param1)
208  {
209      #if 1//defined()
210      {
211          uint32_t x_offset,y_offset;
212          uint32_t w_start,h_start;
213          scale_set_in_out_size(scale3->scale_dev,scale3->iw,scale3->ih,ow,oh);
214
215          w_start = (scale3->iw*10)/atcmd_enlarge;
216          h_start = (scale3->ih*10)/atcmd_enlarge;
217
218          scale_set_step(scale3->scale_dev,w_start,h_start,ow,oh);
219
220          x_offset = (scale3->iw - w_start)/2;
221          y_offset = (scale3->ih - h_start)/2;
222          scale_set_start_addr(scale3->scale_dev,x_offset,y_offset);
223      }
224      #else
225      scale_set_in_out_size(scale3->scale_dev, scale3->iw, scale3->ih, ow, oh);
226      scale_set_step(scale3->scale_dev, scale3->iw, scale3->ih, ow, oh);
227
228      //scale_set_step(scale3->scale_dev, scale3->iw, scale3->ih, 1280, 720);
229      #endif
230      scale_set_out_yaddr(scale3->scale_dev, (uint32_t) p_buf);
231      scale_set_out_uaddr(scale3->scale_dev, (uint32_t) p_buf + ow * oh);
232      scale_set_out_vaddr(scale3->scale_dev, (uint32_t) p_buf + ow * oh + ow * oh / 4);
233  }

```

scale_set in
原图输入size



scale_step out
实际缩放输出size



Note: 需要注意对齐问题，scale 输出 size 遵循 8 字节对齐原则。

12. SCALE2 buf 空间申请规则介绍

如果遇到 JPG 解码放大到屏幕显示异常，比如条纹，花屏幕，需要调整 scale2 buf 空间，放大倍数越大，所需 scale2buf 空间越大

Y 需要的 SRAM 缓存空间

$0x20 + TW + J0 * SRAMBUFLen * 4 + 256$

U 或者 V 需要的 SRAM 缓存空间

$0x10 + TW/2 + J1 * SRAMBUFLen * 2 + 256$

公式参数介绍：

TW-》是 scale2 输出的目标宽度

J0-》是根据放大倍数计算的到的 Y 所需空间系数

输入是 MJPEG YUV420: $J0 = (16 * \text{倍数}) + 2$, 假如 2 倍放大 $J0 = 16 * 2 + 2 = 34$

输入是 H264 YUV420: $J0 = (20 * \text{倍数}) + 2$, 假如 2 倍放大 $J0 = 20 * 2 + 2 = 42$

J1-》是根据放大倍数计算得到的 U 或者 V 所需空间系数

输入是 MJPEG YUV420: $J1 = (8 * \text{倍数}) + 2$, 假如 2 倍放大 $J1 = 8 * 2 + 2 = 18$

输入是 H264 YUV420: $J1 = (12 * \text{倍数}) + 2$, 假如 2 倍放大 $J1 = 12 * 2 + 2 = 26$

256, 128-》分别是冗余处理, 看 sram 紧张程度选择是否添加

Note:如果是存在既要解码 MJPEG 又要解码 H264 建议以 H264 的最大系数去申请 scale2buf。

13. OSD 层透明色设置

- 1) osd 层透明度设置, lcdc_set_osd_alpha 接口范围 0~0x100, 0x00 完全透明到 video 层。

```
482 // 芯片lcd的驱动初始化
483 void lcd_driver_init(const char *osd_encode_name, const char *lcd_osd_name, const char *lcd_video_p0, const char *lcd_video_p1)
484 {
485     #if 1
486     #endif
487     #if 1
488     #endif
489     lcdc_set_osd_start_location(lcd_dev, lcdstruct.osd_x, lcdstruct.osd_y);
490     lcdc_set_osd_size(lcd_dev, lcdstruct.osd_w, lcdstruct.osd_h);
491     lcdc_set_osd_format(lcd_dev, OSD_RGB_565);
492     // 仅仅测试新框架
493     // lcdc_set_osd_dma_addr(lcd_dev, (uint32)osd565_encode_test);
494     lcdc_set_osd_alpha(lcd_dev, 0x100);
495     lcdc_set_osd_enc_cfg(lcd_dev, 0xFFFFFFFF, 0x00000000);
496     lcdc_osd_spec_color_alpha(lcd_dev, 0, 0xffffffff, 0x00);
```

- 2) osd 整层透明色配置, lcdc_set_osd_enc_cfg 接口, 目前设置为 0x000000 为黑色, osd 层中遇这个颜色就会透到 video 层, 多用于 video 层显示。

```
482 // 芯片lcd的驱动初始化
483 void lcd_driver_init(const char *osd_encode_name, const char *lcd_osd_name, const char *lcd_video_p0, const char *lcd_video_p1)
484 {
485     #if 1
486     #endif
487     #if 1
488     #endif
489     lcdc_set_osd_start_location(lcd_dev, lcdstruct.osd_x, lcdstruct.osd_y);
490     lcdc_set_osd_size(lcd_dev, lcdstruct.osd_w, lcdstruct.osd_h);
491     lcdc_set_osd_format(lcd_dev, OSD_RGB_565);
492     // 仅仅测试新框架
493     // lcdc_set_osd_dma_addr(lcd_dev, (uint32)osd565_encode_test);
494     lcdc_set_osd_alpha(lcd_dev, 0x100);
495     lcdc_set_osd_enc_cfg(lcd_dev, 0xFFFFFFFF, 0x00000000);
496     lcdc_osd_spec_color_alpha(lcd_dev, 0, 0xffffffff, 0x00);
497     // lcdc_osd_enc_en(lcd_dev, 1);
498     lcdc_osd_enc_en(lcd_dev, 1);
499     lcdc_set_osd_en(lcd_dev, 0);
500     lcdc_osd_trans_en(lcd_dev, 1);
```

- 3) 特殊颜色做透明色, lcdc_osd_spec_color_alpha 接口, 可以多指定一种颜色做透明色, 并设置透明度, 可以用于做半透菜单栏效果。参数分别是 (设备句柄, 使能, 透明色, 透明度)

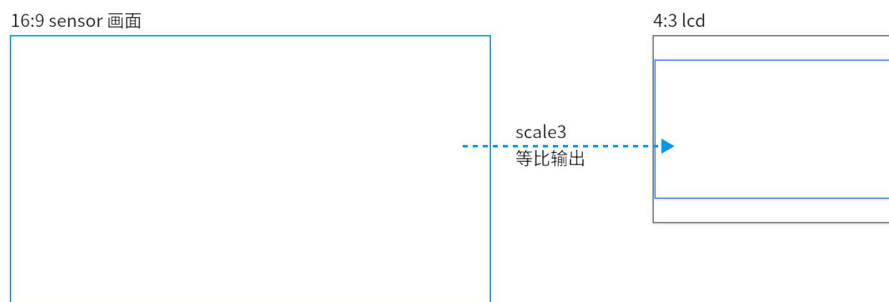
```

482 // 芯片lcd的驱动初始化
483 void lcd_driver_init(const char *osd_encode_name, const char *lcd_osd_name, const char *lcd_video_p0, const char *lcd_video_p1)
484 {
485     #if 1
486     #endif
487     #if 1
488     #endif
489     #endif
490     lcdc_set_osd_start_location(lcd_dev, lcdstruct.osd_x, lcdstruct.osd_y);
491     lcdc_set_osd_size(lcd_dev, lcdstruct.osd_w, lcdstruct.osd_h);
492     lcdc_set_osd_format(lcd_dev, OSD_RGB_565);
493     // 仅仅测试新框架
494     // lcdc_set_osd_dma_addr(lcd_dev, (uint32_t)osd565_encode_test);
495
496     lcdc_set_osd_alpha(lcd_dev, 0x100);
497     lcdc_set_osd_enc_cfg(lcd_dev, 0xFFFF, 0x000000);
498     lcdc_osd_spec_color_alpha(lcd_dev, 0, 0xffff, 0x00);
499     // lcdc_osd_enc_en(lcd_dev, 1);
500     lcdc_osd_enc_en(lcd_dev, 1);
501     lcdc_set_osd_en(lcd_dev, 0);
502     lcdc_osd_trans_en(lcd_dev, 1);
503
504

```

14. VIDEO 层插黑操作

屏幕的宽高比和 sensor 镜头比例不一致的时候，又不想裁剪损失画面，这里就常用到一种手段就是插黑了，如下图 16:9 显示到 4:3 lcd 上下插入黑色，推荐修改 P0 size 和 P0 location 实现。



以 preview_ui 为例子需要设置 scale3_normal_msi 输出 size，以及发送 fb 的时候添加 p0 偏移坐标信息，最终作用到的是 lcd_msi 中 p0 size 和 p0 坐标

```

main_ui.c
lv_obj_t *main_pocket_camera_ui(lv_obj_t *base_ui, lv_group_t
{
    lv_obj_t *btn;
    // 可以打印系统一些信息
    // btn = system_msg_ui(group, base_ui);
    // 预览dvp的镜头数据
    btn = preview_ui(group, base_ui, UI_WIDTH, UI_HEIGHT);
    btn = takephoto_ui(group, base_ui, UI_WIDTH, UI_HEIGHT);
    btn = mp4_player_ui(group, base_ui, UI_WIDTH, UI_HEIGHT);
    btn = mp4_record_ui(group, base_ui, UI_WIDTH, UI_HEIGHT);
    return base_ui;
}

extern struct msi *scale3_normal_msi2(const char *name, uint8
lv_obj_t *main_ui(lv_obj_t *base_ui)
{
    lv_group_t *group;
    group = lv_group_create();
    lv_indev_set_group(indev, keypad, group);
    lv_obj_t *ui = lv_list_create(lv_scr_act());
    // lv_obj_add_style(ui, &g_style, 0);
    lv_obj_set_size(ui, LV_PCT(100), LV_PCT(100));
    switch(OFFLINE_UI)
    {
        case MINI_DV_UI:
            main_Mini_DV_ui(ui, group);
            break;
        case IPC_UI:
            break;
        case BBM_UI:
            lv_obj_add_style(ui, &g_style, 0);
            break;
        case CHILDREN_UI:
            break;
    }
}

scale3_normal_msi.c
static int32 scale3_normal_msi_work(struct os_work *work)
{
    struct scale3_normal_msi *scale3 = (struct scale3_normal_msi *) work;
    struct scale_device *scale_dev = scale3->scale_dev;
    uint8_t delay_time = 0;
    struct framebuffer *fb = NULL;
    struct takephoto_yuv_arg_s *arg = NULL;
    uint16_t ow = 0;
    uint16_t oh = 0;
    int32_t err = -1;
    // 先去检查是否有需要生成额外的yuv数据没
    // 检查如果没有extern_fb, 并且有额外命令, 则生成一个fb
    if (scale3->extern_fb_ready && !scale3->extern_fb)
    {
        fb = (struct framebuffer *) os_msgq_get2(&scale3->msgq, 0, &err);
        if (fb)
        {
            if (fb->datatype != (uint8_t) ~0)
            {
                arg->yuv_arg.x = 80;
                arg->yuv_arg.y = 80;
                arg->yuv_arg.type = YUV_ARG_NONE;
                msi_output_fb(scale3->msi, fb);
            }
            else
            {
                msi_output_fb(scale3->msi, fb);
            }
        }
        fb = NULL;
    }
}

```